

## Code-Layout, Readability and Reusability

|                      |                                 |
|----------------------|---------------------------------|
| <b>Date</b>          | 4 July 2026                     |
| <b>Team ID</b>       | XXXXXX                          |
| <b>Project Name</b>  | Credit Card Approval Prediction |
| <b>Maximum Marks</b> | 5 Marks                         |

### Code-Layout, Readability and Reusability:

Our Credit Card Approval Prediction Framework is developed using structural modularity principles designed to maximize maintainability. By separating raw feature processing pipelines from statistical model scoring engines, the codebase ensures that individual functions—such as null-value imputation, scaling, and classification evaluations—can be tested or scaled independently. Adherence to industry standards ensures that future engineering expansions can integrate new variables or financial logic without needing to refactor the baseline architecture.

### Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes / No / Partial) | Remarks                                                                                                             |
|------|---------------------------|-----------------------------------------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used Throughout the code | Yes                           | Standard PEP 8 spacing is strictly enforced across all script files.                                                |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                           | Separated cleanly into application data (data/), script execution structures (src/), and virtual runtimes (.venv/). |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                           | Explicit descriptive names utilized (e.g., credit_score, income_level, is_approved).                                |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                           | Clear imperative actions mapped out (e.g., preprocess_data(), train_model(), predict_risk()).                       |
| 5    | Code Comments             | Inline and block comments explain logic       | Yes                           | Docstrings cover function signatures, parameter boundaries, and evaluation logic.                                   |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                           | Data cleanup steps and algorithm evaluation tasks run in disconnected steps.                                        |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                           | Dead logic and legacy diagnostic print lines removed entirely post-validation.                                      |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Yes                           | Try-except code tracks missing dataset dependencies and data conversion problems cleanly.                           |

**Reusable Components / Modules:**

| S.No | Component / Module Name     | Language / Technology      | Where Reused                                                                                                   | Reusability Level (High / Medium / Low) |
|------|-----------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| 1    | Data Preprocessing Pipeline | Python / pandas            | Reused in both the model training script and the real-time API request processing function.                    | High                                    |
| 2    | Model Evaluation Engine     | Python / scikit-learn      | Reused when validating and comparing different classifier configurations (XGBoost, Random Forest).             | High                                    |
| 3    | Flask Prediction Controller | Python / Flask             | Can be dropped into any generic financial tabular prediction API layout with minor adjustments.                | Medium                                  |
| 4    | Database Connection Manager | Python / Built-in File I/O | Reused across training data validation steps, audit trails, and inference tracking logging modules.            | High                                    |
| 5    | Feature Exception Handler   | Python / Standard Library  | Reused within all data entry verification forms to intercept invalid datatypes and missing records gracefully. | Medium                                  |

**Overall Code Quality Assessment:**

| Aspect                   | Rating (1-5) | Comments                                                                                                           |
|--------------------------|--------------|--------------------------------------------------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | Repository maps out clean borders separating environment settings, historical datasets, and executable scripts.    |
| Readability              | 5            | Variable choices tell you exactly what properties they track without relying on extra documentation context.       |
| Reusability              | 4            | Feature manipulation routines treat incoming raw parameters agnostic of training or API prediction boundaries.     |
| Documentation / Comments | 4            | Core math and data structural operations are well documented with clear docstrings.                                |
| Overall Score            | 4.5          | High-quality, production-ready codebase built using logical design principles that simplify maintenance over time. |